Hardware-Software Interface
Coursework 1 - Image Processing in C

Adam Sampson
School of Mathematical and Computer Sciences, Heriot-Watt University

Overview

In this coursework, you will demonstrate your ability with the C programming language and its standard library by completing a C program that performs image processing operations on image files.

This coursework contributes to the following learning outcomes of the course:

- Ability to develop efficient, resource-conscious code.
- Practical skills in low-level, systems programming, with effective resource management.
- Ability to articulate system-level operations and to identify performance implications of given systems.

This is an **individual** assessment. The code and documentation you submit must be entirely your own work.

If you have any questions, please ask your local course leader.

Changelog

This section will list any clarifications we have made in response to student questions.

(None at present.)

Finding your tasks

This assessment is **personalised** for each student, so different students in the class are doing slightly different tasks. The tasks are chosen deterministically for each student based on a hash of their student number and the semester.

To find your tasks, see the student task list in **Coursework 1** on Canvas. You will find an entry like this for your student number:

```
H12345678: HQDEC REFLECT NORM
```

Note this entry down in your `README.md` file before you start work. It is important that you check you are solving the correct tasks — if you do an incorrect task, we can't give you any points for it.

The problem

This coursework involves writing a simple RGB bitmap image processing program in C, called `process`. `process` is run from the command line with two arguments, an input image file (which must already exist) and an output image file (which will be created or overwritten). For example:

```
./process kitten.image kitten-processed.image
```

All image files read or written by this program must be in your personalised image format; see below.

The image format

An RGB colour bitmap image consists of a grid of pixels, each of which has red, green and blue colour values. Image files consist of a header, describing the properties of the image, followed by the RGB data for the pixels of the image.

The image format is personalised; check your list of tasks and read the appropriate section below.

Task HQ8

An HQ8 image has an ASCII text header:

```
HQ8
width height nvalues
```

where:

- `HQ8` — fixed code indicating HQ8 format
- `width` — integer number of columns
- `height` — integer number of rows
- `nvalues` — integer number of values per pixel

For all the images used in this coursework, `nvalues` will be 3. If your program reads a file where `nvalues` is not 3, it should report it as an error.

The fields in the header are separated by one or more whitespace (space, tab, carriage return or newline) characters. The final `width` field is followed by a single whitespace character.

The image data follows the header. Each pixel is stored as three unsigned **8-bit** values as **binary data** (not text). Pixels are stored from left to right in the first row, then left to right in the second row, and so on until the end of the image.

Task HQ16

An HQ16 image has an ASCII text header:

```
HQ16
width height nvalues
```

where:

- `HQ16` — fixed code indicating HQ16 format
- `width` — integer number of columns
- `height` — integer number of rows
- `nvalues` — integer number of values per pixel

For all the images used in this coursework, `nvalues` will be 3. If your program reads a file where `nvalues` is not 3, it should report it as an error.

The fields in the header are separated by one or more whitespace (space, tab, carriage return or

newline) characters. The final `width` field is followed by a single whitespace character.

The image data follows the header. Each pixel is stored as three unsigned **16-bit** values as **binary data** in native byte order (not text). Pixels are stored from left to right in the first row, then left to right in the second row, and so on until the end of the image.

(**Native byte order** means that bytes within a number are stored in the same order that they would be stored in memory on the machine you're running the program on.)

Task HQDEC

An HQDEC image consists entirely of ASCII text. It starts with a header:

```
HQDEC
width height nvalues
```

where:

- `HQDEC` — fixed code indicating HQDEC format
- `width` — integer number of columns
- `height` — integer number of rows
- `nvalues` — integer number of values per pixel

For all the images used in this coursework, `nvalues` will be 3. If your program reads a file where `nvalues` is not 3, it should report it as an error.

The image data follows the header. Each pixel is stored as three unsigned **8-bit** values as **decimal numbers**. Pixels are stored from left to right in the first row, then left to right in the second row, and so on until the end of the image.

All fields in the file are separated by one or more whitespace (space, tab, carriage return or newline) characters.

Task HQHEX

An HQHEX image consists entirely of ASCII text. It starts with a header:

```
HQHEX
width height nvalues
```

where:

- `HQHEX` — fixed code indicating HQHEX format
- `width` — integer number of columns
- `height` — integer number of rows
- `nvalues` — integer number of values per pixel

For all the images used in this coursework, `nvalues` will be 3. If your program reads a file where `nvalues` is not 3, it should report it as an error.

The image data follows the header. Each pixel is stored as three unsigned **16-bit** values as **hexadecimal numbers** (digits A-F may be lower or upper case). Pixels are stored from left to right in the first row, then left to right in the second row, and so on until the end of the image.

All fields in the file are separated by one or more whitespace (space, tab, carriage return or

newline) characters.

Getting started

You will need a Linux C development environment with the GNU toolchain, like we have been using in the practical exercises. This might be:

- a Raspberry Pi
- a Linux lab machine
- a MACS machine remotely, using x2go
- a Linux virtual machine on your own computer
- a regular Linux installation on your own computer

We **do not** recommend working on a Windows, WSL or macOS system — they have differences in library and filesystem behaviour that will cause problems for this coursework.

You may like to refer to Hans-Wolfgang Loidl's Linux Introduction.

Setting up the project

Download `cwk1-c-2025.zip` from **Coursework 1** on Canvas. This contains a template project containing an outline source file.

Make sure that you can compile the starter code before you start making changes to the `process.c` file. Change into the project directory, and run:

```
make
```

(This will compile, but will print some warnings because the code is incomplete.)

The template project also includes a Python program, `hqconvert`, that can convert standard PPM images to and from the HQ formats, or view an HQ image using `gpicview` (an image viewer that is often pre-installed on the Raspberry Pi; if it is not installed on your Linux system already, you can install it yourself). For help, run it as:

```
./hqconvert --help
```

A collection of image files for you to test your program with, in all the formats above, is also available from **Coursework 1** on Canvas as `cwk1-images-2025.zip`.

Requirements

This project as a whole is marked out of **20 points**, and makes up **40%** of your final mark for the course.

The program overall should be valid C99 (or later), and should be robust against error. You may write additional helper functions and/or type declarations if you like, but you should keep the functions that are provided in the template file in the existing order.

Work through the steps below in order.

Q1 - (1 point) README.md

The `README.md` is a short file in Markdown or plain text format, saying clearly how to compile your program, and how to run it from the command-line showing the arguments that it takes.

It is **not** a report: you should not restate the problem or write a description of how your program works here.

Q2 - (2 points) Representing images

In `process.c`, complete the declaration of `struct Image`, which holds an image read from a file.

This will need to include the information from the image header (`height`, `width`, `nvalues`), and a pointer to a dynamically-allocated array of `struct Pixel`s for the pixel data. Your program should be able to deal with image files with arbitrary numbers of rows and columns.

You should change the data types inside `struct Pixel` to match your image format. If your image format stores 16-bit colour values, your processing code needs to use data types with a precision of (at least) 16 bits to avoid destroying information.

Q3a - (1 point) Freeing image objects

Complete the `free_image` function, which should free a `struct Image` and its contents.

Q3b - (2 points) Loading images

Complete the `load_image` function, which allocates a new `struct Image` and reads the contents of an image file (in your personalised format) into it.

You will need to use the `fscanf` and/or `fread` functions to read data from the file. Once you know `height` and `width`, construct a dynamic array of the appropriate size. Be careful to handle errors — the input file might not contain valid contents.

Q3c - (2 points) Saving images

Complete the `save_image` function, which writes the contents of a `struct Image` out to an image file (in your personalised format).

You should test this function by using it to write out an image loaded with `load_image`. You can modify `main` temporarily for this.

Q3d - (2 points) Duplicating image objects

To do Q3, you will need to allocate a new `struct Image` that is a copy of an existing image. Complete the `copy_image` function to do this. It will need to copy both the `struct Image` and the pixel data within it.

Q4 - (4 points) Personalised task 1

This task is personalised; check your list of tasks and read the appropriate section. All of these tasks should return a copy of the original image — they must not modify the original image.

Task BLUR

The `apply_BLUR` function should apply a 5x5 blur to an image. Each red colour value in the output image should be computed as the mean of the red values of the 5x5 block of pixels surrounding it in the input image (i.e. a 5x5 square of pixels centred on the input pixel, including

the input pixel itself). The blue and green values should be computed the same way.

Be careful to avoid accessing locations outside the bounds of the image.

Task MEDIAN

The `apply_MEDIAN` function should apply a 3x3 median filter to an image. Each red colour value in the output image should be computed as the median of the red values of the corresponding input pixel and the eight pixels immediately adjacent to it (i.e. a 3x3 square of pixels centred on the input pixel, including the input pixel itself). The blue and green values should be computed the same way.

To compute the median of a set of numbers, sort them into order and take the middle one. For example, you could copy the values into a fixed-length array, then call the `qsort` function from the standard library to sort them.

Be careful to avoid accessing locations outside the bounds of the image.

Task REFLECT

The `apply_REFLECT` function should sum an image with a copy of itself reflected vertically (i.e. the top of the image becomes the bottom of the image, and vice versa). Each red colour value in the output image should be computed as the sum of the red value of the original pixel, and the red value of the corresponding pixel in the reflected image. The blue and green values should be computed the same way.

As you are adding two arbitrary colour values together, you may end up with a result that is too large to be a valid colour value. In this case, the output value should be the maximum possible value.

You do not need to store a reflected copy of the image in memory — you can just compute the position of the corresponding reflected pixel.

Task BRIGHT

The `apply_BRIGHT` function should multiply the red, green and blue values of each pixel by a floating-point factor. If you multiply by a factor greater than 1, it will make the image brighter; if you multiply by a factor smaller than 1, it will make the image dimmer.

First, make it multiply each value by the constant 0.7, and test that this makes the image dimmer.

Then modify the `main` function so that the program takes an extra command-line argument at the end of the command line to indicate the brightness factor. If you run the program as:

```
./process in.image out.image 1.3
```

it should multiply by 1.3. You can use the `atof` or `strtod` standard library functions to convert a string argument to a floating-point value.

As you are multiplying by an arbitrary factor, you may end up with a result that is too small or too large to be a valid colour value. In these cases, the output value should be the minimum or maximum possible value respectively.

Q5 - (4 points) Personalised task 2

This task is personalised; check your list of tasks and read the appropriate section. All of these

functions should print their output to `stdout` (e.g. using `printf`).

Task NORM

The `apply_NORM` function should first scan through the image to find the smallest and largest values used in the red, green and blue values (all together; you don't need to track red/green/blue separately). It should print a message like this to stdout:

```
Minimum value: 10
Maximum value: 255
```

It should then normalise the image by scaling all the values so that the minimum value becomes 0, and the maximum value becomes the biggest possible value that can be stored. Do this in two steps: subtract an offset to bring the minimum to 0, then multiply by a scaling factor to bring the maximum to the largest value.

Because this means modifying the image, you will need to remove the `const` from the type of `apply_NORM`'s image argument.

Task HIST

The `apply_HIST` function should compute separate histograms of the red, green and blue values used in the image, showing how many times each value was used in the image. It should print a report in this format to stdout:

```
Red value 0: 123 pixels
Red value 2: 8729 pixels
Red value 255: 17 pixels
Green value 0: 45 pixels
Green value 127: 78 pixels
Green value 255: 2001 pixels
Blue value 0: 1 pixels
Blue value 255: 400 pixels
```

It should not print a line for any value that did not occur in the image (i.e. it should never say `0 pixels`).

You will need to use a data structure such as an array to count how many times you've seen each value.

Task EDGE

The `apply_EDGE` function should report on the strength of the vertical edges within each horizontal row of the image. For each row in the image, it should scan through the pixels in that row, computing the difference in red, green and blue values between horizontally-adjacent pixels (e.g. computing the difference for all three colour values between 0 and 1, then 1 and 2, then 2 and 3 and so on). It should keep track of the smallest and largest differences found within each row (using a single value for all three colours together; you don't need to track red/green/blue separately).

It should print a report in this format to stdout, with one line of output for each row in the image, and ending with a summary giving the smallest and largest differences found across the whole image:

```
Row 0: minimum 10, maximum 78
Row 1: minimum 0, maximum 255
...
Overall: minimum 0, maximum 255
```

Be careful to avoid accessing locations outside the bounds of the image.

Task COMP

The `apply_COMP` function should take two `struct Image *` arguments, and it should count how many pixels are different between the two images. Two pixels are considered different if any of the three colour values within them are different.

It should print a report in this format to stdout:

```
Identical pixels: 1827841 (68%)
Different pixels: 871285 (32%)
Total pixels: 2699126
```

All the numbers printed should be integers.

You will need to modify the `main` function so that it takes a single extra image argument at the start of the command line to compare with, e.g.:

```
./process reference.image input.image output.image
```

Q6 - (2 points) Processing multiple input files

You should complete all of the tasks above before attempting this one.

Make the program work for an arbitrary number of input/output image filenames rather than just a single one. It must load all of the images into memory before doing any processing.

You will need to use a data structure such as a linked list or a dynamically-allocated array of `struct Image` pointers to hold the list of images you are working on. You could add a field for the output filename to `struct Image`.

Submission

Submission is due through the **Coursework 1** assignment on Canvas. Please check the assignment details on Canvas to see your submission deadline — different campuses may have different deadlines.

You must submit a `.zip` file containing the project files, including at least:

- `process.c` — the source code for your program
- `README.md` — file explaining how to compile and run your program

To reduce the submission size, please don't include the sample images or other large unnecessary files in your submission.

Demonstration

You must give a demonstration of your code to one of the teaching staff during one of the lab sessions. This lets us see that your code works, and lets us ask you questions about your code. Please bring your source code and have a compiled version of the program ready to demonstrate.

Times for demonstrations will be published on Canvas near the submission date.

If you don't give a demonstration, you **will not** receive a mark for this coursework. If you are unable to attend the lab for this (e.g. because of illness) then please contact your local course leader to make alternative arrangements.

Recommendations

Here are some suggestions to help improve the quality of your implementation. Better-quality code will result in higher marks, so you should follow these recommendations when completing the tasks above.

- Your program should be as efficient as possible — particularly when processing image data. You should document any decisions you have made that improve your program's efficiency in the comments for each section of code.
- Your program should also be robust — for example, it shouldn't crash when given an incorrect or corrupt input file, or if the computer runs out of memory. Remember to check the return values of the standard library functions you are using and do something sensible if they fail.
- You are working with data types that can represent a limited range of values (e.g. unsigned integers). When operating on pixel RGB values, be careful not to go out of the valid range of values — this would look bad in the resulting image.
- Be careful about memory leaks (forgetting to `free` memory that you have allocated dynamically), array bounds violations and other types of memory error. You may like to use a dynamic analysis tool such as Address Sanitizer or Valgrind to check for these kinds of errors in your program.
- Try to avoid unnecessary repetition in your code. For example, many of the tasks require you to do the same thing for three different colour values. If possible, you should use a loop or a helper function rather than writing three similar pieces of code.
- The template code has various `TODO` comments in to indicate the things you need to change. Make sure you remove these comments as you complete the tasks — there shouldn't be any left in the submitted program, so you can see if you've forgotten any tasks.

Academic misconduct

Your attention is drawn to the university policy on academic misconduct and to the Contract Cheating Risk and Avoidance Guide (available on Canvas).

The code you submit must be your own work. If some text or code in the coursework has been taken from other sources (beyond the material provided for this course), you must give a clear reference to the source in comments in the code or the `README` file. We cannot give any marks for work that is not your own.

Failure to clearly reference work that has been obtained from other sources, or copying the words or code of another student, is plagiarism.

Asking a third party to complete coursework on your behalf, and then submitting it as your own work, is contract cheating.

You must never give a copy of your coursework writing or code to another student, and you must always refuse any request from another student for a copy of your coursework. Sharing your coursework with another student is collusion.

Suspected plagiarism, contract cheating or collusion will be referred to the School Discipline Committee for investigation. This will delay your feedback, and if you are found guilty of